SQLModel

SQLModel, SQL databases in Python, designed for simplicity, compatibility, and robustness.

Test passing | Publish failing | coverage 99% | pypi package v0.0.37

Documentation: https://sqlmodel.tiangolo.com

Source Code: https://github.com/fastapi/sqlmodel

SQLModel is a library for interacting with SQL databases from Python code, with Python objects. It is designed to be intuitive, easy to use, highly compatible, and robust.

SQLModel is based on Python type annotations, and powered by Pydantic and SQLAlchemy.

The key features are:

- **Intuitive to write**: Great editor support. Completion everywhere. Less time debugging. Designed to be easy to use and learn. Less time reading docs.
- **Easy to use**: It has sensible defaults and does a lot of work underneath to simplify the code you write.
- **Compatible**: It is designed to be compatible with **FastAPI**, Pydantic, and SQLAlchemy.
- **Extensible**: You have all the power of SQLAlchemy and Pydantic underneath.
- **Short**: Minimize code duplication. A single type annotation does a lot of work. No need to duplicate models in SQLAlchemy and Pydantic.

Sponsors

GOVCERT .lu

SQL Databases in FastAPI

FastAPI

SQLModel is designed to simplify interacting with SQL databases in FastAPI applications, it was created by the same author. 😁

It combines SQLAlchemy and Pydantic and tries to simplify the code you write as much as possible, allowing you to reduce the **code duplication to a minimum**, but while getting the **best developer experience** possible.

SQLModel is, in fact, a thin layer on top of **Pydantic** and **SQLAlchemy**, carefully designed to be compatible with both.

Requirements

A recent and currently supported version of Python.

As **SQLModel** is based on **Pydantic** and **SQLAlchemy**, it requires them. They will be automatically installed when you install SQLModel.

Installation

Make sure you create a virtual environment, activate it, and then install SQLModel, for example with:

```bash
$ pip install sqlmodel
100%
Successfully installed sqlmodel
```

Example

For an introduction to databases, SQL, and everything else, see the SQLModel documentation.

Here's a quick example. ✨

A SQL Table

Imagine you have a SQL table called `hero` with:

- `id`
- `name`
- `secret_name`
- `age`

And you want it to have this data:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null

id	name	secret_name	age
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

Create a SQLModel Model

Then you could create a **SQLModel** model like this:

```
from sqlmodel import Field, SQLModel


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None
```

That class `Hero` is a **SQLModel** model, the equivalent of a SQL table in Python code.

And each of those class attributes is equivalent to each **table column**.

Create Rows

Then you could **create each row** of the table as an **instance** of the model:

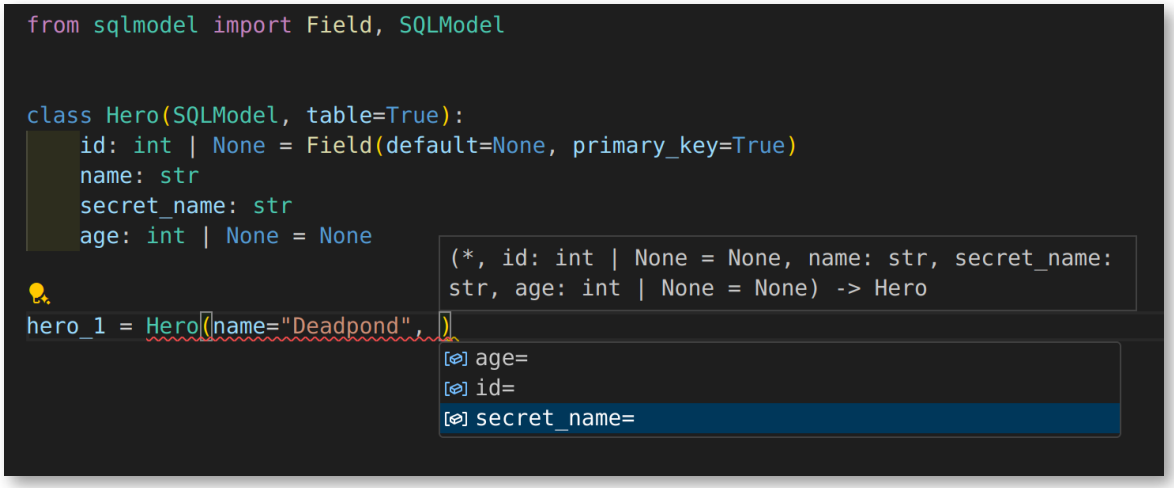
```
hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
```

This way, you can use conventional Python code with **classes** and **instances** that represent **tables** and **rows**, and that way communicate with the **SQL database**.

Editor Support

Everything is designed for you to get the best developer experience possible, with the best editor support.

Including **autocompletion**:



And **inline errors**:

```
from sqlmodel import Field, SQLModel


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
Operator "+" not supported for types "str" and "Literal[5]" Pylance(reportOperatorIssue)
Expression value is unused Pylance(reportUnusedExpression)
(variable) name: str
View Problem (Alt+F8)    Quick Fix... (Ctrl+.)    Fix using Copilot (Ctrl+I)
hero_1.name + 5
```

Write to the Database

You can learn a lot more about **SQLModel** by quickly following the **tutorial**, but if you need a taste right now of how to put all that together and save to the database, you can do this:

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)


engine = create_engine("sqlite:///database.db")


SQLModel.metadata.create_all(engine)

with Session(engine) as session:
    session.add(hero_1)
    session.add(hero_2)
    session.add(hero_3)
    session.commit()
```

That will save a **SQLite** database with the 3 heroes.

Select from the Database

Then you could write queries to select from that same database, for example with:

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


engine = create_engine("sqlite:///database.db")

with Session(engine) as session:
    statement = select(Hero).where(Hero.name == "Spider-Boy")
```

```
    hero = session.exec(statement).first()
    print(hero)
```

Editor Support Everywhere

SQLModel was carefully designed to give you the best developer experience and editor support, **even after selecting data** from the database:

```
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


engine = create_engine("sqlite:///database.db")


with Session(engine) as session:
    statement = select(Hero).where(Hero.name == "Spider-Boy")
    hero = session.exec(statement).first()
    print(hero.)
                name
                age
                Config
                construct
                copy
                dict
                from_orm
                id
                json
                metadata
                model_computed_fields
                model_config
```

SQLAlchemy and Pydantic

That class `Hero` is a **SQLModel** model.

But at the same time, ✨ it is a **SQLAlchemy** model ✨. So, you can combine it and use it with other SQLAlchemy models, or you could easily migrate applications with SQLAlchemy to **SQLModel**.

And at the same time, ✨ it is also a **Pydantic** model ✨. You can use inheritance with it to define all your **data models** while avoiding code duplication. That makes it very easy to use with **FastAPI**.

License

This project is licensed under the terms of the MIT license.